

JavaScript Interview Questions & Answers

56 essential questions with explanations, code samples, and outputs — covering fundamentals, arrays & objects, closures & scope, async JavaScript, the DOM, and coding challenges.

A complete revision guide for JavaScript technical interviews

Page 1 — JavaScript Fundamentals

1. What is JavaScript?

ANSWER

JavaScript is a high-level, dynamically typed programming language mainly used to make web pages interactive. It runs in browsers and can also run on servers using environments such as Node.js.

```
console.log("Hello, JavaScript!");
```

JavaScript can be used for:

Web development

Frontend applications

Backend development

Mobile applications

Desktop applications

APIs and automation

2. What are the differences between var, let, and const?

Feature	var	let	const
Scope	Function	Block	Block
Reassignment	Yes	Yes	No
Redeclaration	Yes	No	No
Hoisting	Yes	Yes, but TDZ	Yes, but TDZ

```
var name = "John";
let age = 25;
const country = "India";

age = 26;           // Valid
// country = "USA"; // Error
```

Interview tip: Prefer let and const in modern JavaScript. Use const by default when reassignment isn't needed.

3. What is the difference between == and ===?

`==` performs type conversion before comparison, while `===` checks both value and type.

```
console.log(5 == "5");    // true
console.log(5 === "5");   // false
```

Another example:

```
console.log(false == 0);  // true
console.log(false === 0); // false
```

Interview answer: "=== is generally preferred because it avoids unexpected type coercion."

4. What are primitive data types in JavaScript?

The main primitive types are:

- String
- Number
- BigInt
- Boolean
- Undefined
- Null
- Symbol

```
let name = "Rahul";    // String
let age = 25;           // Number
let active = true;      // Boolean
let value;              // Undefined
let data = null;        // Null
let big = 123456789n;    // BigInt
let id = Symbol("id");   // Symbol
```

5. What is typeof?

typeof is an operator used to determine the type of a value.

```
console.log(typeof "Hello"); // string
console.log(typeof 100);      // number
console.log(typeof true);     // boolean
console.log(typeof undefined); // undefined
```

One famous JavaScript behavior:

```
console.log(typeof null);
```

OUTPUT

```
object
```

This is a historical behavior of JavaScript.

6. What is hoisting?

Hoisting is JavaScript's behavior where declarations are processed before code execution.

```
console.log(x);  
  
var x = 10;
```

OUTPUT

```
undefined
```

Conceptually, it behaves roughly like:

```
var x;  
  
console.log(x);  
  
x = 10;
```

With let and const:

```
console.log(x);  
let x = 10;
```

This results in a `ReferenceError` because the variable is in the Temporal Dead Zone (TDZ) until its declaration is executed.

7. What is a function?

A function is a reusable block of code.

```
function add(a, b) {  
    return a + b;  
}  
  
console.log(add(10, 20));
```

OUTPUT

30

8. What is an arrow function?

Arrow functions provide shorter function syntax.

```
const add = (a, b) => {  
  return a + b;  
};
```

For a single expression:

```
const add = (a, b) => a + b;  
  
console.log(add(5, 3));
```

OUTPUT

8

9. Function declaration vs function expression

Function declaration:

```
function greet() {  
  console.log("Hello");  
}
```

Function expression:

```
const greet = function () {  
  console.log("Hello");  
};
```

Function declarations are hoisted differently from function expressions assigned to variables.

10. What are template literals?

Template literals use backticks and allow embedded expressions.

```
const name = "Amit";  
const age = 25;
```

```
console.log(`My name is ${name} and I am ${age} years old.`);
```

OUTPUT

```
My name is Amit and I am 25 years old.
```

Page 2 — Arrays, Objects & Modern JavaScript

11. What is an array?

An array stores multiple values in an ordered collection.

```
const fruits = ["Apple", "Banana", "Mango"];

console.log(fruits[0]);
```

OUTPUT

```
Apple
```

12. Difference between `map()`, `filter()`, and `reduce()`

`map()` — Creates a new array by transforming every element.

```
const numbers = [1, 2, 3, 4];

const squares = numbers.map(num => num * num);

console.log(squares);
```

OUTPUT

```
[1, 4, 9, 16]
```

`filter()` — Returns elements that satisfy a condition.

```
const numbers = [1, 2, 3, 4, 5, 6];

const even = numbers.filter(num => num % 2 === 0);

console.log(even);
```

OUTPUT

```
[2, 4, 6]
```

`reduce()` — Reduces an array to a single value.

```
const numbers = [1, 2, 3, 4];
```

```
const sum = numbers.reduce((total, num) => {
  return total + num;
}, 0);

console.log(sum);
```

OUTPUT

```
10
```

Interview tip: map → transform | filter → select | reduce → accumulate

13. What is an object?

An object stores data as key-value pairs.

```
const user = {
  name: "Rahul",
  age: 25,
  city: "Guwahati"
};

console.log(user.name);
```

OUTPUT

```
Rahul
```

You can also use bracket notation:

```
console.log(user["city"]);
```

14. What is destructuring?

Destructuring allows values to be extracted from arrays or objects.

Object destructuring

```
const user = {
  name: "Rahul",
  age: 25
};

const { name, age } = user;
```

```
console.log(name);  
console.log(age);
```

Array destructuring

```
const colors = ["red", "green", "blue"];  
  
const [first, second] = colors;  
  
console.log(first); // red  
console.log(second); // green
```

15. What is the spread operator?

The spread operator `...` expands iterable values.

```
const arr1 = [1, 2, 3];  
const arr2 = [4, 5, 6];  
  
const result = [...arr1, ...arr2];  
  
console.log(result);
```

OUTPUT

```
[1, 2, 3, 4, 5, 6]
```

It can also be used with objects:

```
const user = {  
  name: "Amit",  
  age: 25  
};  
  
const updatedUser = {  
  ...user,  
  age: 26  
};  
  
console.log(updatedUser);
```

16. What is the rest operator?

The rest operator collects remaining values.

```
function sum(...numbers) {  
  return numbers.reduce((total, n) => total + n, 0);  
}  
  
console.log(sum(1, 2, 3, 4));
```

OUTPUT

```
10
```

Important: Spread and rest both use `...`, but their purpose depends on where they are used.

17. What are default parameters?

Default parameters provide fallback values.

```
function greet(name = "Guest") {  
  console.log(`Hello ${name}`);  
}  
  
greet();  
greet("Rahul");
```

OUTPUT

```
Hello Guest  
Hello Rahul
```

18. What is optional chaining?

Optional chaining `?.` safely accesses nested properties.

```
const user = {  
  profile: {  
    name: "Rahul"  
  }  
};  
  
console.log(user.profile?.name);  
console.log(user.address?.city);
```

OUTPUT

```
Rahul  
undefined
```

Without optional chaining, accessing a missing intermediate object can cause an error.

19. What is nullish coalescing?

The `??` operator returns the right-hand value when the left-hand value is null or undefined.

```
const username = null;  
  
const result = username ?? "Guest";  
  
console.log(result);
```

OUTPUT

```
Guest
```

Compare:

```
console.log(0 || 100); // 100  
console.log(0 ?? 100); // 0
```

Page 3 — Closures, Scope, this & Advanced Concepts

20. What is scope?

Scope determines where a variable can be accessed.

Global scope

```
const name = "Rahul";

function greet() {
  console.log(name);
}

greet();
```

Function scope

```
function test() {
  var x = 10;
  console.log(x);
}

// console.log(x); // Error
```

Block scope

```
{
  let x = 10;
  const y = 20;
}

// x and y aren't accessible here
```

21. What is a closure?

A closure occurs when a function remembers variables from its outer lexical environment even after the outer function has finished executing.

```
function counter() {
  let count = 0;
```

```

    return function () {
        count++;
        return count;
    };
}

const increment = counter();

console.log(increment());
console.log(increment());
console.log(increment());

```

OUTPUT

```

1
2
3

```

The inner function retains access to `count`.

Common interview use cases:

Data privacy

Function factories

Callbacks

Event handlers

Maintaining state

22. What is lexical scope?

Lexical scope means the accessibility of variables is determined by where functions and blocks are written in the source code.

```

const name = "Global";

function outer() {
    const name = "Outer";

    function inner() {
        console.log(name);
    }

    inner();
}

outer();

```

OUTPUT

```

Outer

```

The inner function can access variables from its surrounding lexical environment.

23. What is the this keyword?

`this` generally refers to the object associated with the current function invocation.

```
const user = {
  name: "Rahul",

  greet() {
    console.log(this.name);
  }
};

user.greet();
```

OUTPUT

```
Rahul
```

The exact value of `this` depends on how a function is called.

24. How is this different in arrow functions?

Arrow functions do not create their own `this`. They capture `this` from the surrounding lexical scope.

```
const user = {
  name: "Rahul",

  greet: function () {
    const inner = () => {
      console.log(this.name);
    };

    inner();
  }
};

user.greet();
```

OUTPUT

```
Rahul
```

25. What are call(), apply(), and bind()?

They allow you to control the value of `this`.

call() — Arguments are passed individually.

```
function greet(city) {
  console.log(`${this.name} lives in ${city}`);
}

const user = {
  name: "Rahul"
};

greet.call(user, "Guwahati");
```

apply() — Arguments are passed as an array.

```
greet.apply(user, ["Guwahati"]);
```

bind() — Returns a new function.

```
const newGreet = greet.bind(user, "Guwahati");

newGreet();
```

26. What is a callback function?

A callback is a function passed to another function to be executed later or by that function.

```
function processUser(name, callback) {
  console.log(`Processing ${name}`);
  callback();
}

processUser("Rahul", () => {
  console.log("Completed");
});
```

OUTPUT

```
Processing Rahul
Completed
```

27. What is a higher-order function?

A higher-order function either takes another function as an argument, or returns a function.

```
function calculate(a, b, operation) {  
  return operation(a, b);  
}  
  
const result = calculate(10, 5, (x, y) => x + y);  
  
console.log(result);
```

OUTPUT

```
15
```

28. What is the event loop?

JavaScript is commonly described as single-threaded, but it can handle asynchronous operations using the runtime's event loop and associated queues.

```
console.log("A");  
  
setTimeout(() => {  
  console.log("B");  
}, 0);  
  
console.log("C");
```

OUTPUT

```
A  
C  
B
```

Even with 0 milliseconds, the timer callback doesn't execute immediately. It is scheduled to run later.

Page 4 — Promises, Async/Await & APIs

29. What is a Promise?

A Promise represents the eventual result of an asynchronous operation. It has three major states:

Pending

Fulfilled

Rejected

```
const promise = new Promise((resolve, reject) => {
  const success = true;

  if (success) {
    resolve("Operation successful");
  } else {
    reject("Operation failed");
  }
});

promise
  .then(result => console.log(result))
  .catch(error => console.log(error));
```

30. What is async/await?

`async/await` provides a convenient syntax for working with Promises.

```
function getData() {
  return Promise.resolve("Data received");
}

async function fetchData() {
  const result = await getData();

  console.log(result);
}

fetchData();
```

OUTPUT

```
Data received
```

31. How do you handle errors with async/await?

Use `try...catch`.

```
async function fetchData() {
  try {
    const result = await Promise.reject("Something went wrong");

    console.log(result);
  } catch (error) {
    console.log("Error:", error);
  }
}

fetchData();
```

OUTPUT

```
Error: Something went wrong
```

32. What is Promise.all()?

`Promise.all()` waits for multiple promises to fulfill.

```
const p1 = Promise.resolve("A");
const p2 = Promise.resolve("B");
const p3 = Promise.resolve("C");

Promise.all([p1, p2, p3])
  .then(results => {
    console.log(results);
  });
```

OUTPUT

```
["A", "B", "C"]
```

If one promise rejects, `Promise.all()` rejects.

33. Difference between Promise.all() and Promise.allSettled()

`Promise.all()` fails when one promise rejects.

```
Promise.all([p1, p2, p3]);
```

`Promise.allSettled()` waits for all promises and reports each result.

```
Promise.allSettled([p1, p2, p3])
  .then(results => console.log(results));
```

This is useful when you want the outcome of every operation regardless of individual failures.

34. What is the Fetch API?

The Fetch API is commonly used to make HTTP requests.

```
fetch("https://example.com/api/users")
  .then(response => response.json())
  .then(data => {
    console.log(data);
  })
  .catch(error => {
    console.error(error);
  });
```

Using async/await:

```
async function getUsers() {
  try {
    const response = await fetch(
      "https://example.com/api/users"
    );

    const data = await response.json();

    console.log(data);
  } catch (error) {
    console.error(error);
  }
}

getUsers();
```

35. What is synchronous vs asynchronous JavaScript?

Synchronous — Operations execute sequentially.

```
console.log("First");
console.log("Second");
console.log("Third");
```

OUTPUT

```
First
Second
Third
```

Asynchronous — Some operations complete later.

```
console.log("First");

setTimeout(() => {
  console.log("Second");
}, 1000);

console.log("Third");
```

OUTPUT

```
First
Third
Second
```

36. What is callback hell?

Callback hell happens when many asynchronous callbacks become deeply nested.

```
getUser(function(user) {
  getOrders(user, function(orders) {
    getPayment(orders, function(payment) {
      console.log(payment);
    });
  });
});
```

Promises and `async/await` can make this easier to read:

```
async function process() {
  const user = await getUser();
  const orders = await getOrders(user);
  const payment = await getPayment(orders);

  console.log(payment);
}
```

Page 5 — DOM, Events & Coding Questions

37. What is the DOM?

DOM stands for Document Object Model. The browser represents an HTML document as a tree of objects that JavaScript can interact with.

HTML:

```
<h1 id="title">Hello</h1>
```

JavaScript:

```
const title = document.getElementById("title");

title.textContent = "Hello JavaScript";
```

38. How do you select an element?

```
document.getElementById("title");

document.querySelector(".container");

document.querySelector("#title");

document.querySelectorAll(".item");
```

`querySelector()` returns the first matching element. `querySelectorAll()` returns a collection of all matching elements.

39. What is event bubbling?

When an event occurs on a nested element, it can propagate upward through its ancestors.

```
<div id="parent">
  <button id="child">Click</button>
</div>
```

```
document.getElementById("parent")
  .addEventListener("click", () => {
    console.log("Parent clicked");
  });
```

```
document.getElementById("child")
  .addEventListener("click", () => {
    console.log("Child clicked");
  });
```

Clicking the button can trigger the button handler and then the parent handler through bubbling.

40. What is event delegation?

Event delegation attaches an event listener to a parent rather than adding separate listeners to many children.

```
document
  .getElementById("list")
  .addEventListener("click", event => {
    if (event.target.matches("li")) {
      console.log(event.target.textContent);
    }
  });
```

Benefits include fewer event listeners and convenient handling of dynamically added elements.

Coding Questions

41. Reverse a string

Question: Reverse "JavaScript" without using a reverse library method.

```
function reverseString(str) {
  let result = "";

  for (let i = str.length - 1; i >= 0; i--) {
    result += str[i];
  }

  return result;
}

console.log(reverseString("JavaScript"));
```

OUTPUT

```
tpircSavaJ
```

42. Check whether a string is a palindrome

```
function isPalindrome(str) {
  const reversed = str
    .split("")
    .reverse()
    .join("");

  return str === reversed;
}

console.log(isPalindrome("madam"));
console.log(isPalindrome("hello"));
```

OUTPUT

```
true
false
```

43. Find the largest number in an array

```
function findLargest(numbers) {
  let largest = numbers[0];

  for (const number of numbers) {
    if (number > largest) {
      largest = number;
    }
  }

  return largest;
}

console.log(findLargest([10, 5, 30, 20]));
```

OUTPUT

```
30
```

44. Remove duplicate values from an array

Using `Set` :

```
const numbers = [1, 2, 2, 3, 3, 4];
```

```
const uniqueNumbers = [...new Set(numbers)];  
  
console.log(uniqueNumbers);
```

OUTPUT

```
[1, 2, 3, 4]
```

45. Count character frequency

```
function characterFrequency(str) {  
  const frequency = {};  
  
  for (const char of str) {  
    frequency[char] = (frequency[char] || 0) + 1;  
  }  
  
  return frequency;  
}  
  
console.log(characterFrequency("hello"));
```

OUTPUT

```
{  
  h: 1,  
  e: 1,  
  l: 2,  
  o: 1  
}
```

46. Find even numbers

```
const numbers = [1, 2, 3, 4, 5, 6];  
  
const evenNumbers = numbers.filter(num => num % 2 === 0);  
  
console.log(evenNumbers);
```

OUTPUT

```
[2, 4, 6]
```

47. Find the sum of an array

```
const numbers = [10, 20, 30, 40];

const sum = numbers.reduce(
  (total, num) => total + num,
  0
);

console.log(sum);
```

OUTPUT

```
100
```

48. FizzBuzz

Question: Print numbers from 1 to 20. For multiples of 3 print Fizz, multiples of 5 print Buzz, and multiples of both print FizzBuzz.

```
for (let i = 1; i <= 20; i++) {
  if (i % 15 === 0) {
    console.log("FizzBuzz");
  } else if (i % 3 === 0) {
    console.log("Fizz");
  } else if (i % 5 === 0) {
    console.log("Buzz");
  } else {
    console.log(i);
  }
}
```

Page 6 — Output Questions & Quick Revision

49. What is the output?

```
console.log(a);  
  
var a = 10;
```

ANSWER

```
undefined
```

Because the `var` declaration is hoisted, but its assignment happens later.

50. What is the output?

```
console.log(typeof null);
```

ANSWER

```
object
```

This is a longstanding JavaScript behavior.

51. What is the output?

```
console.log(1 + "2");
```

ANSWER

```
12
```

The number is converted to a string during the operation.

52. What is the output?

```
console.log("5" - 2);
```

ANSWER

```
3
```

The string "5" is converted to a number for subtraction.

53. What is the output?

```
console.log([] == false);
```

This is an example of JavaScript's type coercion rules and can produce:

```
true
```

This is one reason strict equality is generally easier to reason about.

54. What is the output?

```
console.log("A");

setTimeout(() => {
  console.log("B");
}, 0);

Promise.resolve().then(() => {
  console.log("C");
});

console.log("D");
```

OUTPUT

```
A
D
C
B
```

The Promise reaction is processed before the timer callback after the current synchronous code completes.

55. What is debouncing?

Debouncing delays execution until a certain period has passed without another invocation. It is commonly useful for:

Search boxes

Input validation

Window resize events

API requests

```
function debounce(fn, delay) {
  let timer;

  return function (...args) {
    clearTimeout(timer);

    timer = setTimeout(() => {
      fn.apply(this, args);
    }, delay);
  };
}

const search = debounce((value) => {
  console.log("Searching:", value);
}, 500);

search("Java");
search("JavaScript");
```

The function waits until calls stop for the specified delay.

56. What is throttling?

Throttling limits how frequently a function can execute. It is useful for:

Scroll events Mouse movement Resize events Continuous user interactions

Common interview distinction: Debounce — execute after activity stops. Throttle — execute at controlled intervals during activity.

Final JavaScript Interview Cheat Sheet

Core Concepts

var, let, const Primitive vs reference values == vs === Hoisting Temporal Dead Zone Scope
Closures Lexical scope this call, apply, bind

Arrays

map() filter() reduce() find() findIndex() some() every() includes() slice() splice()

Modern JavaScript

const let arrow functions template literals destructuring spread rest optional chaining
nullish coalescing modules

Asynchronous JavaScript

setTimeout() setInterval() Promise async await fetch() Promise.all() Promise.allSettled()

Browser / DOM

document.querySelector() document.querySelectorAll() addEventListener() textContent classList
event.target event bubbling event delegation

Coding Problems to Practice

- Reverse a string
- Check palindrome
- Find largest number
- Find smallest number
- Remove duplicates
- Count character frequency
- Count word frequency
- Find even/odd numbers
- Find prime numbers
- Fibonacci series
- Factorial
- FizzBuzz

- Sort an array
- Find missing number
- Find duplicate number
- Flatten an array
- Two-sum problem
- Find maximum occurring character
- Merge two arrays
- Implement debounce
- Implement throttle
- Implement a basic Promise-style flow
- Deep-clone an object
- Group objects by property
- Implement a simple counter using closure

★ Most Important Questions to Prepare First

If you're preparing for an actual JavaScript interview, make sure you can explain and code these without looking at notes:

- var vs let vs const
- == vs ===
- Hoisting
- Scope
- Closure
- this
- Arrow functions
- call() / apply() / bind()
- map() / filter() / reduce()
- Promises
- async/await
- Event loop
- Microtasks vs tasks
- Event bubbling/delegation
- Debouncing/throttling
- Destructuring
- Spread/rest operators
- Optional chaining
- Promise.all()

- Common coding problems